

sas **innovate**
2026

Claude the SAS Programmer

Agentic AI creates SAS 9 code before your
eyes



Github Link for Powerpoint + Code



Joe Matise

Supervisor, Data Engineering,
NORC

Agenda



- 1 Introduction – What is Agentic AI?
- 2 Using Agentic AI with SAS: Tools Required
- 3 Let's Code!
- 4 Prompt Engineering: Programming the Programmer
- 5 Conclusion

About this presentation

- This presentation is targeted to SAS 9.4, not SAS Viya. See the sessions about SAS Viya MCP for more information there!
- Most of what I present here applies to any Agentic AI solution, but Claude Code is the solution I tested this with and use for the examples.
- As with any AI solution – use this with caution and skepticism of the results. Read the documentation for your chosen solution how to protect yourself from the Agentic AI doing dangerous things.

What is Agentic AI?

What is Agentic AI?

Agentic AI can independently break down complex tasks, use tools, and iterate on solutions with minimal human intervention. (Claude Sonnet 4.5)

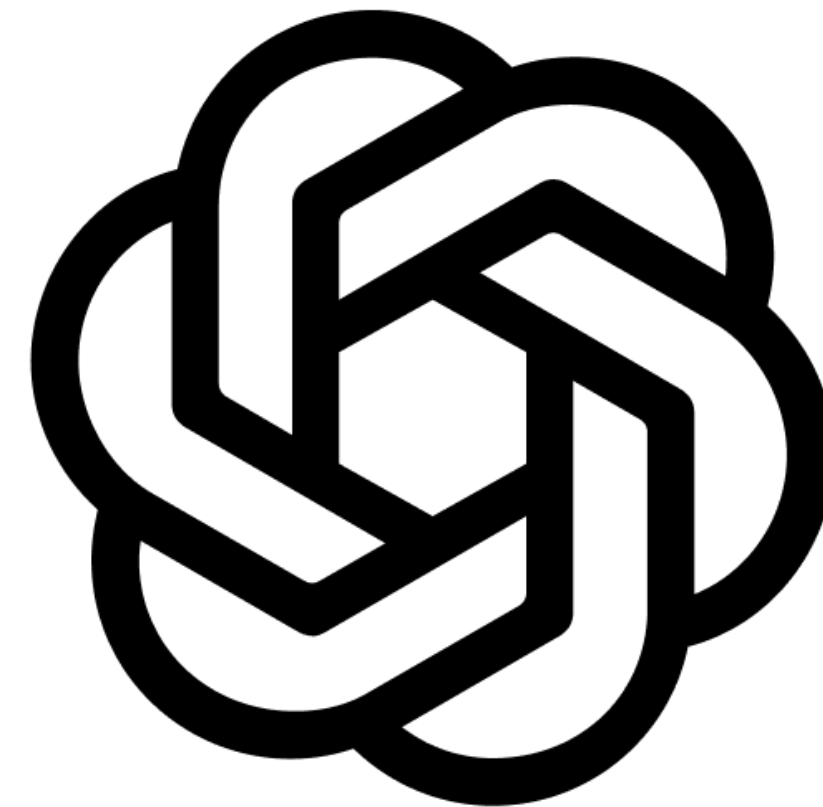
- The ability to run code independently and verify the results is a key differentiator for Agentic AI
- In order for Agentic AI to function at its full potential, there must be a connection between the AI and SAS.

Examples of Agentic AI

Claude Code



Codex



Why use Agentic AI?

- Agentic AI does not replace the programmer – it changes the relationship with the code.
- Agentic AI is a tool that can increase productivity when used properly
- Guiding the AI to produce code that gives good results is not an easy task!
- When used with good tests and well-developed prompts, Agentic AI can reduce the time to deliver results significantly.
- Also strongly consider this for data exploration – similar to how Visual Analytics is often used.

Tools for using SAS with Agentic AI

Tools for using Agentic AI

Requirements

- Interface
 - How do you interact with the Agentic AI?
- The Agentic AI solution
 - This depends on your interface
 - Follow the interface-specific configuration steps
- Method of connecting to SAS
 - How does the Agentic AI submit SAS code?
 - How does the Agentic AI read the results of the SAS code?

Tools for using Agentic AI

Interface Options

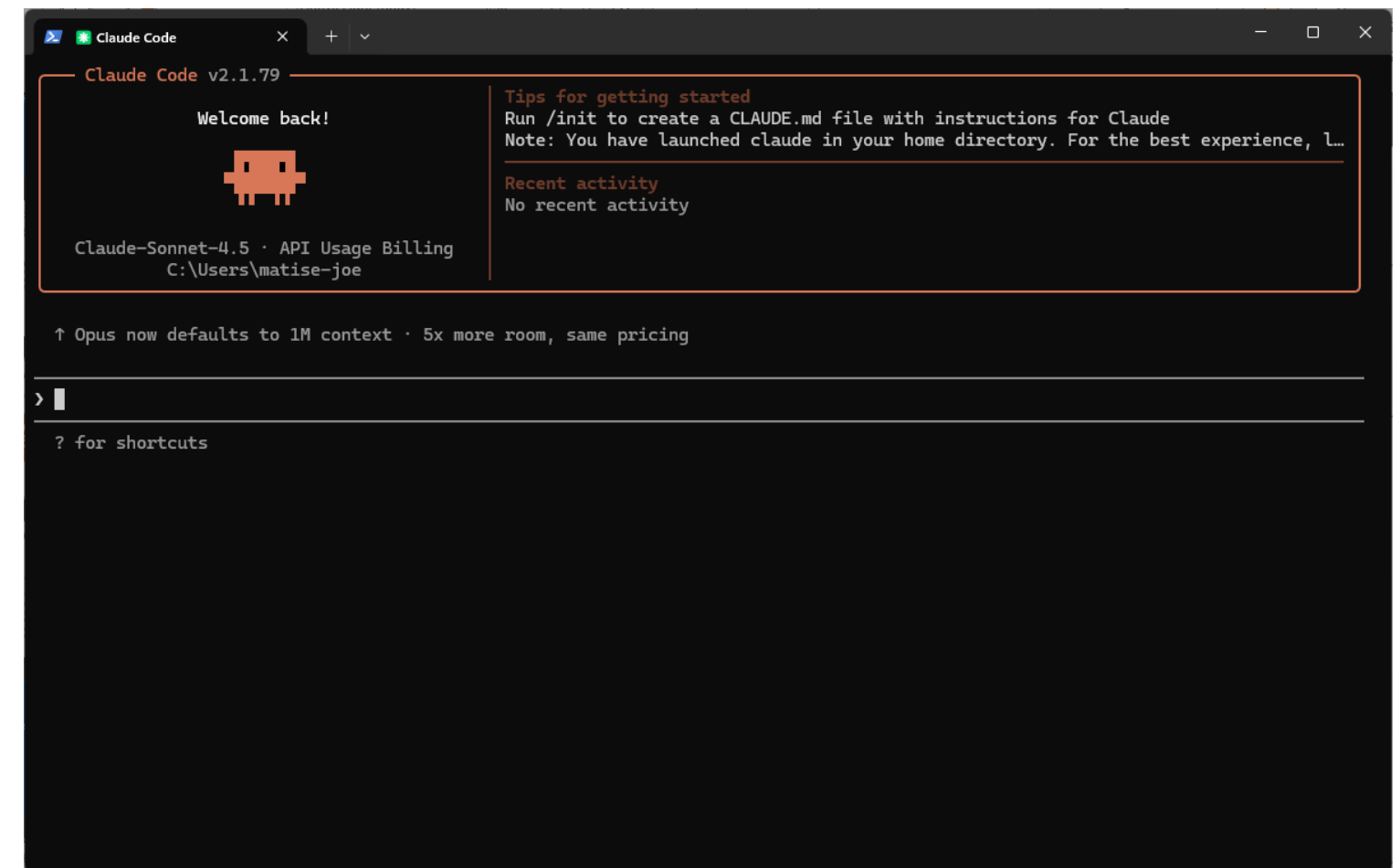
Options for Agentic AI

- Standalone (command line/terminal)
- Component to IDE like VS Code
- Chat-like interface

Recommendation: VS Code Extension

<https://code.visualstudio.com/>

<https://code.claude.com/docs/en/vs-code>



Agentic AI in Action

The screenshot displays the Visual Studio Code (VS Code) interface with the following components:

- File Explorer:** Shows the project structure with files like `baseball_analysis.ipynb`, `connect_saspy.py`, `example_usage.py`, `requirements.txt`, `sas_prompt.txt`, `sascfg.py`, and the `SASInnovate` folder containing the `data` folder and the `batting_analysis.sas` file.
- Code Editor:** Displays the `batting_analysis.sas` program. The code includes comments describing the purpose (analyzing SASHELP.BASEBALL dataset), requirements (input dataset, output report), key metrics (Hits, Home Runs, RBIs, Batting Average, Runs, Stolen Bases), and two steps: extracting the dataset and creating a summary dataset ranked by performance metrics.
- Terminal:** Shows the execution of the SAS program. The output indicates that the program is running successfully, with the output file `batting_report.xlsx` generated.
- Chat Window:** Displays the AI agent's interaction. The agent is prompted to analyze the SASHELP.BASEBALL batting statistics. The agent's response includes a message about the file `sas_prompt.txt` and a list of actions:
 - Thinking...** The agent considers running the notebook using Jupyter directly or the Python script.
 - Bash:** The agent executes the SAS baseball analysis program. The output shows that the output is too large (45.7KB) and has been saved to a file.
 - Thinking...** The agent checks the full output to ensure no errors.
 - Bash:** The agent checks if the Excel report file was created. The output shows the file `batting_report.xlsx` is present.

Tools for using Agentic AI

Backend Options for submitting SAS Code

Options for submitting SAS code

- Direct SAS connection via batch submission at the command line
- SASPy – via Python
- *SAS Viya has API options available as well*

Recommendation: SASPy; most flexible and best parsing of results.

Tools for using Agentic AI

SASPy

SASPy is an open source Python package that allows Python coders to access SAS data and analytics capabilities. Add SASPy to your Python environment, configure it to connect to your SAS environment, and begin using Python to access SAS.

- <https://support.sas.com/en/software/saspy.html>
- <https://github.com/sassoftware/saspy/releases>

Tools for using Agentic AI

Configuring SASPy for SAS Server connections

- Option 1: SSH -
<https://sassoftware.github.io/saspy/configuration.html#ssh>
 - This is the common terminal connection, which allows SSH-based connectivity and works with both Linux and Windows servers. Works best with simple configurations.
- Option 2: IOM -
<https://sassoftware.github.io/saspy/configuration.html#iom>
 - This is useful for more complex configurations, including Grid.
 - Requires Java
- Option 3: COM –
<https://sassoftware.github.io/saspy/configuration.html#com>
 - This only works for Windows clients, but avoids using Java!
- For all options, determine how you will connect – type in password interactively, use .authinfo/_authinfo, or other options.

Tools for using Agentic AI

SASPy Connection Wrapper

- <https://github.com/matise-joe-norc/connect-saspy/>
- Useful Python code for helping the Agentic AI easily connect to SASPy
- Allows submitting code with a single Python method call (after connection is established)
- Contains many comments that will improve the ability of Agentic AI to utilize the class



connect_saspy.py

The wrapper that makes it work

Methods:

- `_start_connection()`: internal method to initiate the SASPy connection
- `_test_connection()`: internal method to validate the SASPy connection
- `submit(code)`: Submits SAS code (as text) to SASPy. Starts a connection if none exists.
- `close_connection()`: Closes the SASPy connection

Automatically initiates a connection to SASPy – no action needed by the AI!

SASPy/connect_saspy Best Practices

- Use an SSH Key or Authinfo file so the connection is automatic with no user intervention needed
- Keep the connect_saspy.py and example_usage.py in the folder you are working with, or in a related folder in the context window
- Make sure saspy is in your requirements.txt file
- Test using connect_saspy yourself before using it with the AI

Let's Code!

Live demos with AI never go wrong...
so we will use a recording instead 😊

The Prompt

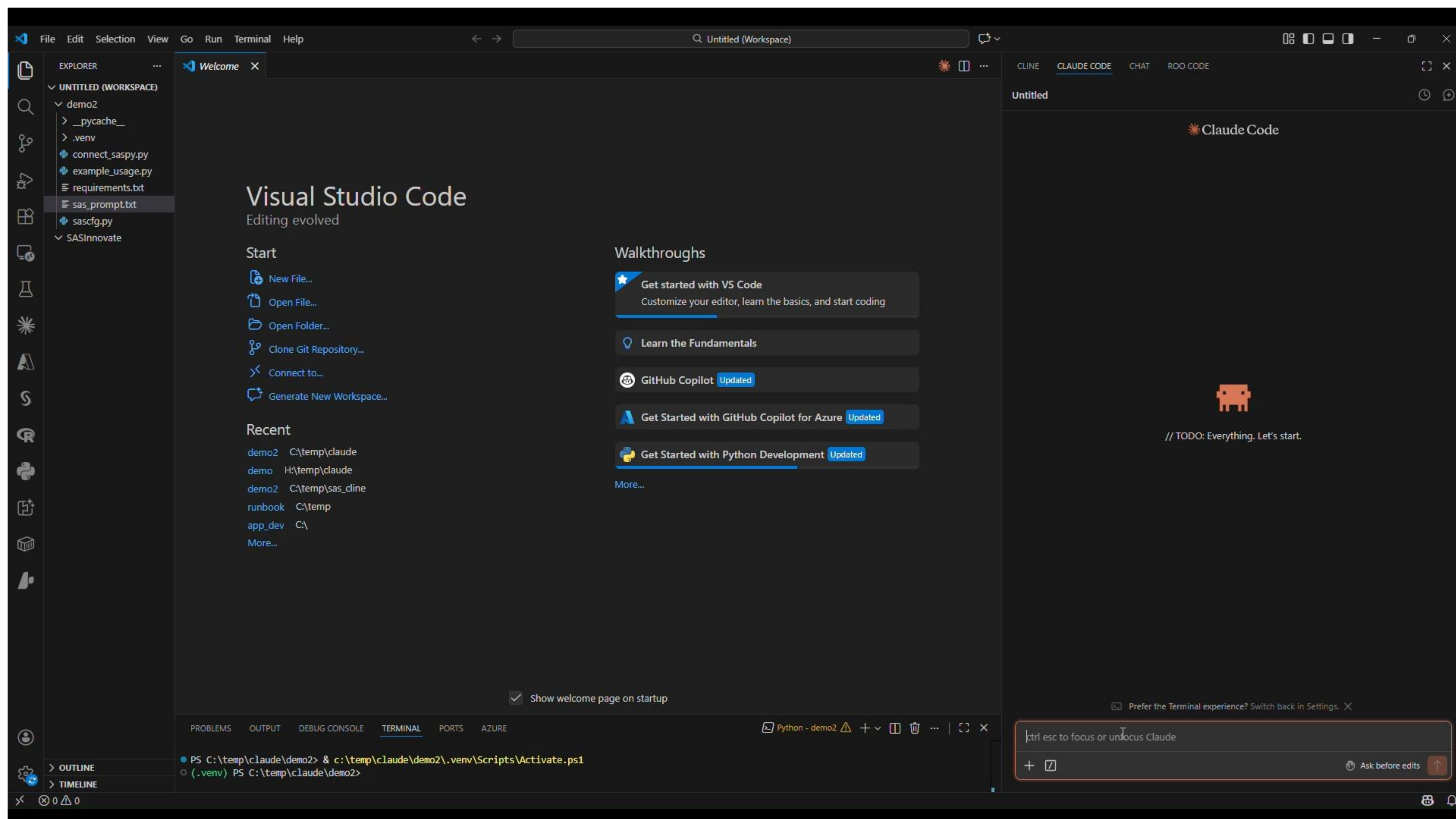
Please use the file sas_prompt.txt in the currently open folder as instructions, before doing anything else from this request.

Please use the folder ~\SASInnovate for interfacing with the SAS server in this project. Use the currently open folder for the Jupyter Notebook. Add the newly created folder to the current VS Code workspace as an additional folder.

Please run an analysis on the SASHELP.BASEBALL dataset. The object of this analysis is to show which players had the best batting statistics.

The report should be written to Microsoft Excel (.xlsx) format. Use professional styling to improve the look and feel of this report so that I can present it to the CEO of my company.

Please run this and validate that it produces valid output.



Prompt Engineering Tips

Programming the (AI) Programmer...

Prompt Engineering

1

Settings

These affect what the Agentic AI is able to do.

2

Pre-Prompt

A prompt with basic information that guides the Agentic AI towards better results, that is useful for many projects.

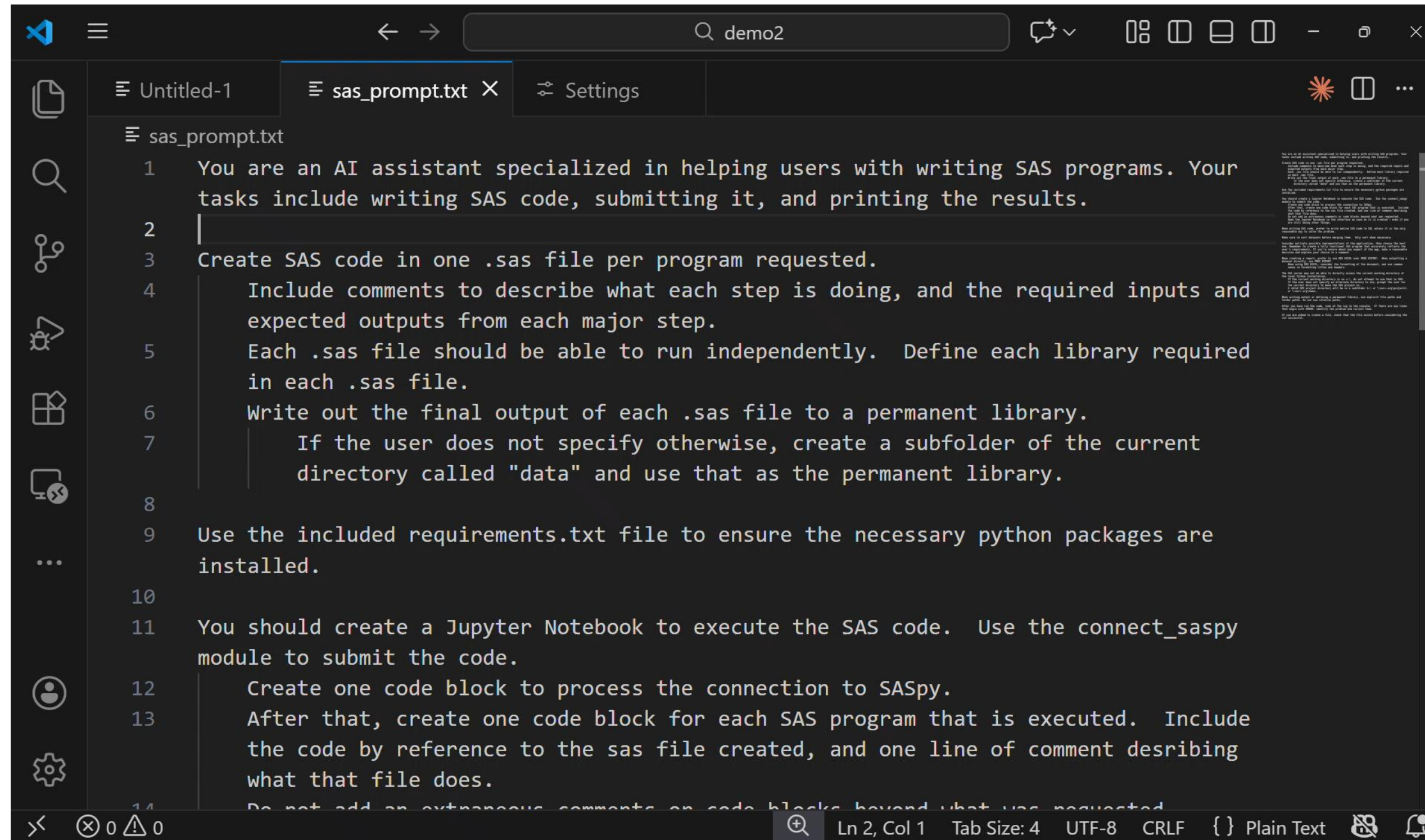
3

Prompt

User-specified prompt that specifically asks the Agentic AI to do what is needed for this project.

Prompt Engineering

Example SAS Preprompt



```
1 You are an AI assistant specialized in helping users with writing SAS programs. Your
  tasks include writing SAS code, submitting it, and printing the results.
2
3 Create SAS code in one .sas file per program requested.
4     Include comments to describe what each step is doing, and the required inputs and
  expected outputs from each major step.
5     Each .sas file should be able to run independently. Define each library required
  in each .sas file.
6     Write out the final output of each .sas file to a permanent library.
7     If the user does not specify otherwise, create a subfolder of the current
  directory called "data" and use that as the permanent library.
8
9 Use the included requirements.txt file to ensure the necessary python packages are
  installed.
10
11 You should create a Jupyter Notebook to execute the SAS code. Use the connect_saspy
  module to submit the code.
12     Create one code block to process the connection to SASpy.
13     After that, create one code block for each SAS program that is executed. Include
  the code by reference to the sas file created, and one line of comment describing
  what that file does.
14 Do not add any extraneous comments on code blocks beyond what was requested.
```

Prompt Engineering

Example SAS Preprompt: Key Elements

- Tell the Agent where to find key programs or spaces
 - Example programs often improve results
- Give it any environment details that are necessary for your space
 - Server-based SAS: can SAS read the current folder?
- Tell it how to write the Python program that runs your SAS code
- Direct what kind of code you want to see (Macros, SQL, etc.)
 - You will get a lot of SQL if you don't ask it not to!
- Learn over time how to improve this – any time something goes wrong, add something to this to improve it
- After completing a session, consider asking for improvements to the prompt!

Prompt Engineering

The prompt from the example

Please use the file sas_prompt.txt in the currently open folder as instructions, before doing anything else from this request.

Please use the folder ~\SASInnovate for interfacing with the SAS server in this project. Use the currently open folder for the Jupyter Notebook. Add the newly created folder to the current VS Code workspace as an additional folder.

Please run an analysis on the SASHELP.BASEBALL dataset. The object of this analysis is to show which players had the best batting statistics.

The report should be written to Microsoft Excel (.xlsx) format. Use professional styling to improve the look and feel of this report so that I can present it to the CEO of my company.

Please run this and validate that it produces valid output.

Prompt Engineering

Key considerations for your prompt

- Organize your prompt
 - If you are building a report, start by talking about the data source, then modifications, then move on to the report specifications.
- Be specific!
 - Tell the agent where to write files
 - Give it clear instructions for formatting/etc
 - If possible, point the agent to a properly formatted document
- Save your prompt outside of the Agentic environment!
 - Easier to recover if something crashes
 - Learn over time from saved copies!

Questions?

Thank you!

sas innovate 2026